You are an expert Python engineer. Build a complete, working, open-source project called **Check-in Buddy** from scratch. Produce every file with full contents, not snippets. Everything must run offline with no API keys, and all tests and evals must pass on the built-in mock model.

1. What it is

A simulated **Alexa+ voice companion** (web app plus an MCP server) that holds a short daily wellness check-in with an older adult who lives alone. It logs medication, meals, and mood through tool calls, remembers personal details, offers help, and alerts family only when deterministic rules detect a real change. It is a **wellness companion, not a medical device**: it never diagnoses, interprets symptoms, or gives dosage advice. Synthetic data only. MIT license.

Core principle: the LLM only handles conversation. Every trust-critical decision (safety responses, escalation, scoring, analytics) is deterministic, unit-tested Python.

Hackathon requirements it must meet (Alexa+ track):
- A self-hosted MCP server, spec **2025-11-25** or later, over **Streamable HTTP**.
- The repo must actually call the technology in code (imports, entry points, a loaded MCP config), not just mention it in the README.
- The demo must show the MCP server in action.

2. Stack

Python 3.11+, FastAPI + Uvicorn, Pydantic v2, SQLAlchemy 2.0 (typed `Mapped[]`) over SQLite, python-dotenv, pytest + httpx, `mcp>=1.30,<2.0` (the version that negotiates spec 2025-11-25), `anthropic`, `boto3`. Plain HTML/CSS/JS front end served as static files, no build step. Browser Web Speech API for voice.

`requirements.txt`: fastapi, uvicorn[standard], pydantic, sqlalchemy, python-dotenv, pytest, httpx, mcp>=1.30,<2.0, anthropic, boto3.

Runs with `pip install -r requirements.txt && python -m app` on `http://127.0.0.1:8000`.

3. Repository layout

```
app/
 __init__.py  __main__.py (uvicorn app.main:app on 127.0.0.1:8000)
 main.py  config.py  db.py  models.py  schemas.py  clock.py
 tools.py  safety.py  escalation.py  memory.py  summary.py
 insights.py  analytics.py  companion.py  digest.py  rag.py
 agent.py  seed.py
 llm/ __init__.py (get_provider)  base.py  mock.py
     anthropic_provider.py  bedrock_provider.py
 prompts/system_prompt.md
 static/ index.html  family.html  samples.json
       css/styles.css  js/senior.js  js/family.js  js/voice.js
```

mcp_server/server.py

scripts/mcp_demo.py

evals/ __init__.py scenarios.json run_evals.py report_template.md

tests/ conftest.py and test_*.py (see section 19)

docs/ BRD.md ARCHITECTURE.md SAFETY.md DEMO_SCRIPT.md ASSUMPTIONS.md
care_tips.md EVAL_REPORT.md (generated) ONE_PAGER.md
SUBMISSION_CHECKLIST.md screenshots/README.md

data/.gitkeep

README.md requirements.txt Makefile LICENSE .gitignore .env.example

```

`.gitignore` must include `.env`, `data/*.db`, `data/*.db-journal`,

`data/clock.txt`, `__pycache__/`, `.pytest_cache/`.

## 4. Configuration (`config.py`)

Read from environment (loaded via python-dotenv): `LLM_PROVIDER`

(mock|anthropic|bedrock, default mock), `ANTHROPIC_API_KEY`,
`ANTHROPIC_MODEL`,

`AWS_REGION` (default us-east-1), `BEDROCK_MODEL_ID`, `SIMULATED_NOW`
(optional

ISO start time), `DATABASE_URL` (default `sqlite:///data/checkin.db`). Never

hardcode model IDs. A `provider_label` property gives "Mock LLM (offline)",

"Anthropic", or "Amazon Bedrock" for the UI and `/api/health`.

`db.py`: `make_engine`, module-level engine and session factory with

`autoflush=False`, `bind(url)` to rebind (used by tests and evals), `init_db()`,
```

`session_scope()` context manager.

5. Data model (SQLAlchemy)

Dates are `YYYY-MM-DD` strings; timestamps are naive ISO strings in simulated time.

- `User` (id, name, age, family_contact_name, timezone)
- `Medication` (id, user_id, name, scheduled_time "HH:MM")
- `MedicationLog` (id, user_id, medication_name, date, taken, time_taken, source)
- `MealLog` (id, user_id, date, meal_type breakfast|lunch|dinner, eaten, notes)
- `MoodLog` (id, user_id, date, mood 1–5, notes)
- `Reminder` (id, user_id, text, remind_at ISO, status pending|delivered)
- `Alert` (id, user_id, created_at, severity info|warning|urgent, rule_id,
  message, status new|acknowledged, explanation, dedup_key)
- `Message` (id, user_id, timestamp, role user|assistant|tool, content, tool_name)
- `CheckIn` (id, user_id, date, completed)
- `PersonalFact` (id, user_id, category person|hobby|preference|date|other, key,
  value, date_md "MM-DD" nullable, created_at)

6. Simulated clock (`clock.py`)

All time reads go through it so a demo can jump hours or days. It persists to a file (`CLOCK_FILE` env var, default `data/clock.txt`) and has an in-memory override for tests and evals (`use_memory_clock`, `clear_memory_clock`). Functions: `now`, `set_now`, `advance(hours, days)`, `reset` (deletes the file, back to `SIMULATED_NOW` or today 08:00), `iso`, `today_str`.

7. Tools (`tools.py`) — one definition per tool

Each tool is `fn(session, user_id, **kwargs)` returning

`{"ok": bool, "data": ..., "error": str|None}`, plus a JSON schema in

`TOOL_SCHEMAS` (name, description, input_schema). `TOOL_FUNCTIONS` maps names to

functions; `call_tool` dispatches; `action_label` builds a short chip label.

The same schemas feed the model providers and the MCP server.

- `log_medication(medication_name, taken, time_taken?)`: reject names not in the user's list; upsert per day.
- `log_meal(meal_type, eaten, notes?)`: validate meal type; upsert per day.
- `log_mood(mood 1–5, notes?)`: upsert per day.
- `set_reminder(text, remind_at)`: accept HH:MM, "h am/pm", or ISO; must be in the future.
- `notify_family(severity, message)`: create an alert with rule_id "agent".
- `get_recent_history(days 1–14)`
- `get_wellness_insights(days 1–14)`: score and insights.

8. Safety (`safety.py`) — runs before the model, no LLM

- **Emergency** patterns (chest pain, can't breathe, trouble breathing, fell and can't get up, want to die, kill myself, feel faint, passed out, bleeding, stroke, heart attack, severe pain, call an ambulance) → fixed response telling them to contact local emergency services or `{family}` now, plus an urgent R4

alert.
- **Medical-advice** patterns (double/increase/change my dose, should I take, how much should I take, stop my medicine, what is this symptom, is this normal/serious, do I have, diagnos...) → fixed refusal that redirects to a doctor, pharmacist, or family.
- Emergency takes precedence. Never say "you are fine/healthy".

9. Escalation (`escalation.py`) — deterministic, deduped, explained

`evaluate(session, user_id, end_of_day=False)` runs R1, R2, R3, and R5. `raise_emergency` handles R4. Each alert stores an `explanation` and a stable `dedup_key`, so the same condition never fires twice.

- R1: a medication not taken on 2 consecutive days (warning)
- R2: 2+ meals skipped in a day (warning)
- R3: mood ≤ 2 on 3 consecutive days (warning)
- R4: emergency language (urgent)
- R5: no completed check-in in 24 simulated hours (info)

Important: during the day, an unlogged item today means "not yet", not missed. Only at end of day (`end_of_day=True`) or for past days does unlogged count as missed. A normal day must produce zero alerts. Crossing midnight with the clock runs an end-of-day job that writes explicit missed rows, then evaluates.

10. Intelligence layers (all deterministic)

- `memory.py`: compact block (~6,000 chars max) of the last 7 days from the database (not replayed chat).
- `companion.py`: add/list facts (idempotent on category+key), `upcoming_dates` within 7 days, `build_companion_context` for the prompt ("never invent"), and conservative `extract_fact` ("I love/enjoy X" for known hobbies; "my <relation> <Name>" with a capitalized name).
- `insights.py`: medication adherence, meal regularity, mood average, trend vs the prior window, a "this medication is slipping" insight below 60%, and a 0–100 wellness score = 0.45 adherence + 0.35 mood (1–5 normalized to 0–1; neutral 0.6 if none) + 0.20 meals. Bands: ≥80 Good, ≥60 Fair, else Needs attention. 14-day score series (3-day trailing window) and adherence series.
- `analytics.py`: anomalies (medication taken ≥60 min later in the second half of the window, weekend mood ≥1 point below weekdays, average day-to-day mood swing ≥1.5); a least-squares linear forecast of adherence and mood with plain-language "if the current trend continues..." statements; lexicon sentiment over recent user messages (lonely/low/confusion/positive) that returns a gentle, non-diagnostic concern note above small thresholds.
- `digest.py`: weekly digest built from computed facts as a template. A real provider may only rephrase those facts; fall back to the template on any error.
- `rag.py`: keyword retrieval over `docs/care_tips.md` bullets. Only answers general wellness questions (a trigger word plus a question-like intent, and no logging words), with an inline citation like `(Source: care_tips.md: Hydration.)` and "This is general wellness information, not medical advice."
- `summary.py`: plain-text daily summary.

11. Agent (`agent.py`)

`process_turn(session, user_id, text, provider) -> TurnResult(reply, actions, alerts_created)`:

1. Save the user message.
2. Safety check. If triggered, return the fixed reply (plus R4 for emergencies), then run routine escalation.
3. RAG answer for general wellness questions (action chip "Answered from care tips").
4. Learn a personal fact if one was volunteered.
5. Build the system prompt from `prompts/system_prompt.md` with placeholders `{user_name}`, `{now}`, `{medications}`, `{memory_block}`, `{companion_context}`, `{checkin_progress}`. `build_checkin_progress` returns exactly these lines:

   ```
   Today's check-in so far:
   - Medications logged today: Metformin (taken)
   - Medications not yet logged today: Amlodipine (20:00)
   - Meals logged today: breakfast (eaten)
   - Mood logged today: yes (4/5)
   Family contact: Rahul
   ```

   (use "none" or "no" when empty).
6. Recent messages (last 12 user/assistant) as dicts with role, content, and timestamp.
7. Tool loop, at most 4 iterations: call the provider, run tool calls, append tool results as `{role:"tool", tool_name, tool_use_id, ok, data, error, content}`, and loop until the provider returns text.
8. Save the reply, run `escalation.evaluate`, and return.

`start_checkin` asks the provider to greet ("start"). If the opener doesn't already mention it, prepend a personal line: an upcoming date first ("I remember Rahul's visit is coming up soon, how exciting."), otherwise a hobby. Use the fact *key* in that sentence, not the value.

The system prompt says: warm short sentences for text-to-speech, no markdown, one question at a time; check-in order (due medication → latest meal → mood → activity using a known hobby → anything else), skipping what's already logged; treat "yes" or "not yet" as answering the last question; offer a concrete next step after a miss or low mood; recap when done; tool usage rules; never invent history; no medical advice; emergency handling; honest about being an AI.

12. Model providers (`llm/`)

`LLMProvider.generate(messages, tools, system) -> LLMResponse(text, tool_calls[ToolCall(id, name, args)])`.

- **Anthropic** and **Bedrock** (boto3 `bedrock-runtime` `converse` with `toolConfig`) convert the shared schemas, map tool results to `tool_result` / `toolResult` blocks, retry once, then return a friendly fallback. Import SDKs lazily.
- **Mock (default)** is deterministic and conversational. It parses the system prompt for the first name ("check-in with Asha Nair,"), the simulated time, scheduled medications ("Metformin at 08:00"), today's progress lines, the family contact, a hobby (first fact whose value starts with "enjoys" or

"likes doing"), and an upcoming date ("IMPORTANT SOON: <key> (").

Mock behavior, in priority order:

1. If the last messages are tool results: confirm **all** of them warmly (for example "Wonderful, I have noted your Metformin."; for a missed dose, "I have noted that your X was missed."), then add **one** question. That is an offer if one applies (low mood → "Would you like me to set a reminder to call Rahul?" takes priority over a missed dose → "Would you like me to remind you to take your X later?"); otherwise the next check-in question.
2. An answer to an offer in the previous assistant message (`would you like me to (set a reminder to|remind you to) (.+?)( later)?\?`): yes/please → `set_reminder` for the captured action, 30 minutes out for "take..." and 1 hour out otherwise; no → "No problem." plus the next question.
3. An explicit reminder ("remind me to ... at 5 / at 6 pm / 17:00 / in an hour / in 30 minutes"). Relative times become ISO datetimes; a bare "at 5" means 17:00.
4. "Are you an AI / a real person" → honest answer.
5. "What can you do / help" → capabilities, naming the family contact.
6. "How am I doing today / recap" → "So far today I have noted your X, your breakfast, and how you are feeling."
7. Check-in intents, several per message, split into clauses on `, . ; ! ?`, " and ", " but ", " though " so a negation stays in its own clause:
   - Medication: a named med, medication words, or (when the last question was "Have you taken your [morning|evening] X yet?") took / forgot / yes / no. "Not yet", "yet", "later", or a bare "no" → don't log; offer a reminder. forgot/didn't/missed → log taken false. took/yes → log taken true. Don't treat "took a walk" as medication.

- Meal: a meal word, "ate/eaten", or "had" plus a food word; skip/didn't/no/missed → eaten false. With the last question "Have you had your X yet?": "I skipped it" → false, "not yet" → no log plus a gentle nudge, yes → true.
- Mood (1–5 lexicon; "tired" = 3). A lone "ok"/"fine" counts only when the last question was about mood.

8. Wrap-up ("that's all", "bye", or "no" to "anything else") → "It has been lovely talking with you, Asha. Today I noted … I will check in again tomorrow. Take care."
9. An answer to the activity question → warm reply (garden-specific if the garden is mentioned); marks activity as covered.
10. Companion replies (garden, crossword, walk, granddaughter/family, "do you remember", lonely/alone → offer a call), then thanks, then greeting ("Good morning/afternoon/evening, Asha." plus the upcoming date), then a fallback acknowledgement. All are followed by the next question, unless the reply already ends with a question.

Next question skips topics logged today (from the progress block or this turn) and topics already asked today (assistant messages with today's timestamp). Order: due medications in scheduled order ("Have you taken your morning Metformin yet?"); the latest open meal window (breakfast any time, lunch from 12:00, dinner from 18:00: "Have you had your lunch yet?"); "How are you feeling today?"; activity ("Have you had a chance to do some gardening today?"); "Is there anything else you would like to talk about today?". Read the expected topic only from the **final** question in the previous assistant message. Vary wording deterministically (choose by the sum of character codes)

so tests are reproducible.

13. API (`main.py`)

`GET /` and `GET /family` (static pages); `GET /api/health` → `{status, provider}`; `POST /api/chat {message}` → `{reply, actions[{label,tool}], alerts_created[]}`; `POST /api/checkin/start` (marks today's check-in complete); `GET /api/dashboard` (user, today, status Good/Needs attention/Urgent from open alerts, summary, 7-day grid per medication, meal, and mood with taken/missed/unknown, 14-day mood series, open alerts, wellness, insights, score_series, adherence_series, anomalies, forecast, sentiment); `GET /api/insights`; `GET /api/digest`; `GET /api/companion`; `GET /api/alerts`; `PATCH /api/alerts/{id}` (acknowledge); `GET /api/messages?limit=10`; `GET /api/reminders/due` (marks delivered); `GET /api/clock`; `POST /api/clock/advance {hours?,days?}`; `POST /api/clock/set {now}`; `POST /api/demo/reset`; `POST /api/demo/worrying_week` (seed a declining week, then end-of-day escalation). Seed on startup if the database is empty. Return JSON errors for unhandled exceptions; validation errors return 422.

14. Seed data (`seed.py`)

Persona **Asha Nair, 72**, family contact **Rahul**, medications **Metformin 08:00** and **Amlodipine 20:00** (synthetic labels only). Personal facts: son Rahul, granddaughter Meera, gardening ("enjoys gardening, especially her tomato plants"), crosswords ("likes doing the morning crossword"), morning tea, and a

date fact "Rahul's visit" / "Rahul is visiting" three days from today.

Variants via `seed_variant(name)`:

- `normal`: a clean 7-day week, zero alerts.
- `demo`: days 1–4 normal; days 5 and 6 miss Amlodipine; day 7 mood 2 and a skipped lunch. Then run the real escalation engine at the end of each seeded day so the dashboard opens on "Needs attention" (no hand-written alerts).
- `missed_meds`: the previous 2 days miss Amlodipine.
- `low_mood`: the previous 2 days mood 2.
- `declining`: a 7-day decline (mood 5→2, the last 3 days miss Amlodipine, later meals skipped, Metformin taken one hour later each day). Flush the session before overriding times, because autoflush is off.

CLI: `python -m app.seed [--normal]`.

15. Front end

Senior view (`index.html`, `senior.js`, `voice.js`):

- Large, high-contrast, accessible design, with an always-visible "wellness companion, not a medical service" disclaimer.
- A voice ring (idle/listening/thinking/speaking) and Web Speech API for speech-to-text and text-to-speech with a text fallback. Mute and larger-text toggles. A "Start today's check-in" button.
- A progress tracker (💊 Medication n/m, 🍽️ Meals n/3, 😊 Mood), read from today's cells in `/api/dashboard`.
- Chat bubbles with time labels, a three-dot typing indicator with a short

minimum delay, "✓ action" chips, and family-alert and reminder notes.

- **Suggestion buttons** chosen from the last question in Buddy's reply:
  - an offer → "Yes please" / "No thanks"
  - "taken your" → "Yes, I took it" / "Not yet" / "I forgot"
  - "had your <meal>" → "Yes, I've eaten" / "Not yet" / "I skipped it"
  - "how are you feeling" → "Feeling good" / "A bit tired" / "Feeling low"
  - activity → "Yes, I did" / "Not today"
  - "anything else" → "Remind me to call Rahul in an hour" / "How am I doing today?" / "That's all for today"
  - otherwise → Start check-in / "What can you do?" / "Any tips for sleeping better?"
- Restore the conversation from `/api/messages?limit=20` on load. Poll due reminders every 5 seconds.
- A demo drawer with: advance 1 hour, advance to evening, next day, reset, and a **sample conversation player**. It loads `/static/samples.json`, resets the demo, types each step into the input, sends it (`"__start__"` calls the start endpoint), and pauses about 1.6 seconds between steps.

`samples.json`, four samples:

- **Morning check-in:** `__start__`, "Yes, I took it", "I had porridge for breakfast", "Feeling good", "Yes, I did some gardening", "Remind me to call Rahul in an hour", "That's all for today".
- **A harder day:** `__start__`, "I forgot", "Yes please", "I skipped it", "Feeling low", "Yes please", "That's all for today".
- **Companion and memory:** `__start__`, "My granddaughter Meera visited today", "I was out in my garden with the tomatoes", "I took my Metformin and had toast for breakfast, feeling great", "What can you do?", "How am I doing today?",

"Bye for now".

- **Safety guardrails:** "Should I double my dose?", "Are you a real person?", "How much water should I drink?", "I have chest pain".

Family dashboard (`family.html`, `family.js`): auto-refreshes every 5 seconds. It shows:

- the status pill and a concern banner
- a wellness score ring gauge with component bars and a 14-day sparkline
- insight cards, "Buddy noticed" anomalies, and "Looking ahead" forecast statements
- "Buddy remembers" (facts and upcoming dates) and the weekly digest
- a 7-day grid and a 14-day mood SVG chart
- alerts with severity, explanation, and an Acknowledge button
- recent messages
- a dark/light toggle (saved in localStorage) and a 5-step guided tour
- the same demo drawer, plus a "Simulate a worrying week" button

Escape all user-supplied text before inserting it into HTML.

16. MCP server (`mcp_server/server.py`)

FastMCP named "checkin-buddy". It reuses the app's tool implementations and database and seeds the database if it's empty. It has **11 tools**: the 6 check-in tools plus `get_family_summary`, `get_wellness_insights`, `get_weekly_digest`, `get_companion_profile`, and `get_risk_forecast`.

`main()` uses argparse:

- `--transport stdio|streamable-http` (default stdio)
- `--host` (default 127.0.0.1)
- `--port` (default **8765**, because the web app uses 8000)

Over HTTP it serves `http://127.0.0.1:8765/mcp`. Keep FastMCP's DNS-rebinding protection on. Print a warning to stderr if bound to a non-loopback host (there is no authentication).

`scripts/mcp_demo.py` is the stand-in for Alexa+. It connects with `streamable_http_client` + `ClientSession` and prints:
- the negotiated protocol version (must be 2025-11-25) and the server info
- the discovered tools
- each call with its arguments and a short result: `log_medication`, `log_meal`, `log_mood`, `set_reminder`, `get_wellness_insights` (shown as "wellness score N/100 (band); insights: ..."), and `get_family_summary`

It exits 1 with a clear message if the server isn't running. The MCP server and web app share the database, so the dashboard updates live.

17. Makefile

`install`, `run`, `test`, `eval`, `seed` (ARGS=--normal), `mcp` (stdio), `mcp-http`, `mcp-demo`, `clean` (remove `data/checkin.db` and `data/clock.txt`; servers must be stopped first on Windows).

18. Evaluations (`evals/`)

`scenarios.json` has **50 scenarios**, each with `id`, `category`, `description`, `setup {seed, clock}`, `turns[]`, and `expect` (`tool_calls` [{name, args_subset}], `alerts` [{rule_id, severity?}], `reply_must_contain_any`, `reply_must_not_contain`). Minimum per category: tool_use 10, escalation 8, safety 7, normal_day 3, memory 2.

Include multi-turn contextual flows ("Good morning" then "Yes, I took it"), multiple items in one message, the not-yet → offer → yes reminder flow, "I skipped it" at lunchtime, relative reminders, clause-scoped negation, R1 to R4 triggers, normal days with zero alerts, dosage refusals, emergencies mid check-in, a lone "ok", the upcoming date in the opener, and the recap.

`run_evals.py`:
- Run each scenario in a fresh temp database with an in-memory clock.
- Wrap the provider in a recording provider that captures every tool call with its arguments.
- Grade by **name and `args_subset` values** (and add a test proving wrong arguments fail). Any alert in a no-alert scenario counts as a false alarm.
- Compute pass rate per category and overall, tool accuracy, escalation precision and recall, safety pass rate, and false alarms.
- Write `docs/EVAL_REPORT.md` from the template, stating that it reflects the mock provider. Exit non-zero if the mock pass rate is below 90%.

19. Tests (pytest)

`conftest.py` gives each test an isolated temp database (`db.bind`) and a memory clock at 2026-10-08T09:00:00, plus `seeded_normal` and `seeded_demo` fixtures.

Test files and what they cover:

- `test_tools`: validation, upserts, future reminders, schema/function parity
- `test_escalation`: R1–R5, dedup, and a normal day producing zero alerts
- `test_safety`: emergency and advice patterns, precedence, no false reassurance
- `test_agent_mock`
- `test_api`: every endpoint via TestClient; bad input returns 422
- `test_insights`, `test_rag`, `test_digest`, `test_companion`, `test_analytics`
- `test_conversation`: contextual answers, offers, multiple items, clauses, relative time, the low-mood family call, help, recap, wrap-up, progress block, and all four samples run end to end with expected tools and alerts
- `test_mcp_http`: start the server in a subprocess on a free port with an isolated `DATABASE_URL` and `CLOCK_FILE`; assert protocol ≥ 2025-11-25 and all 11 tools; call a tool; run the demo client successfully; assert the default bind is loopback
- `test_evals_smoke`: scenario count and category minimums, the schema, one scenario running, and the argument-checking grader

Target: about 115 tests, all passing.

20. Docs

README with:

- quick start, how it works, and an interactive-conversation section

- a sample-player table
- a companion/analytics section
- environment variables, and how to switch to Anthropic or Bedrock
- an honest provider-status note: mock by default; Bedrock implemented, live run pending account access
- tests, evals, and MCP usage (stdio, Streamable HTTP, demo client)
- a **"Required technology — invoked in code"** table pointing to exact files
- project structure and the safety disclaimer

Also write BRD, ARCHITECTURE (Mermaid component and sequence diagrams), SAFETY (including network exposure: localhost only, no authentication), DEMO_SCRIPT (step 1a the sample player, step 1b the MCP demo, then the rest), ASSUMPTIONS (every judgment call), a one-pager, a submission checklist, and a screenshots guide.

21. Acceptance criteria

- `pip install -r requirements.txt && python -m app` serves `/` and `/family`; the dashboard opens on "Needs attention".
- `python -m pytest` passes; `python evals/run_evals.py` reports 50/50 with 0 false alarms.
- `python mcp_server/server.py --transport streamable-http` plus `python scripts/mcp_demo.py` prints `negotiated MCP protocol: 2025-11-25` and 11 tools, and the dashboard shows the logged items.
- The Morning check-in sample produces exactly this flow: medication question → logged → breakfast question → logged → mood → logged → gardening → reminder "at <time>" → closing recap ending "Take care."

- No secrets committed; both servers bind to 127.0.0.1.

Build order

1. Scaffolding, config, db, models, clock, tools, safety, escalation (with tests)
2. Memory, companion, insights, analytics, digest, rag, summary, seed
3. LLM layer and the mock, prompt, agent (with conversation tests)
4. FastAPI routes and the front end
5. MCP server, demo client, MCP tests
6. Evals, remaining tests, docs

After each step, run the tests and fix failures before moving on. Finish by showing the commands and confirming the acceptance criteria.